

Interação com APIs

09

2026

SENAI - CRTI



Boas Práticas e Criptografia

Segurança profissional na troca de dados

Situação de Aprendizagem

Um Portal Corporativo em Risco

Uma empresa desenvolveu um portal corporativo que integra login, dados pessoais, relatórios, operações financeiras, histórico de alterações e APIs internas e externas. Uma auditoria identificou falhas críticas que colocam em risco toda a operação.



● Logs Insuficientes

Sem registro adequado de ações, é impossível rastrear o que ocorreu.

● Acesso Excessivo

Usuários com permissões além do necessário para suas funções.

● Armazenamento Inseguro

Dados sensíveis gravados sem criptografia ou hash adequado.

● Troca Sem Proteção

Informações sensíveis trafegando sem criptografia em trânsito.

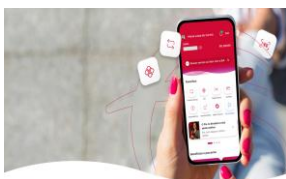


Pergunta-problema: Como construir integrações seguras, auditáveis e alinhadas às boas práticas profissionais?

Contextualização Profissional

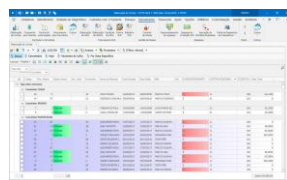
Segurança Está em Todo Lugar

Boas práticas de segurança não são exclusividade de grandes corporações. Estão presentes em qualquer sistema que processe dados de pessoas reais — e desenvolvedores são os responsáveis por garantir essa proteção.



Bancos e Fintechs

Transações, autenticação forte, tokens e criptografia end-to-end.



Hospitais e Saúde

Prontuários, resultados de exames e dados clínicos sob sigilo total.



Governo Digital

CPF, certidões, benefícios — dados de cidadãos exigem máxima proteção.



E-commerce e ERPs

Dados financeiros, histórico de compras e integração com pagamentos.

i Mensagem-chave: Segurança não é recurso opcional — é requisito de qualidade profissional.

Conceito Fundamental

O que é Criptografia?

Criptografia é o processo de proteger informações por meio de transformações matemáticas, tornando os dados ilegíveis para qualquer pessoa que não possua a chave de acesso. É o pilar da confidencialidade digital.

Objetivos principais:

- Confidencialidade: somente partes autorizadas leem os dados
- Privacidade: informações pessoais ficam protegidas
- Proteção contra interceptação: dados inúteis se capturados

Texto Legível

"senha123"

Cifrado

"aX7#kL9!mZ"

Chave

Decodifica para leitura autorizada

Simétrica vs. Assimétrica

🔑 Criptografia Simétrica

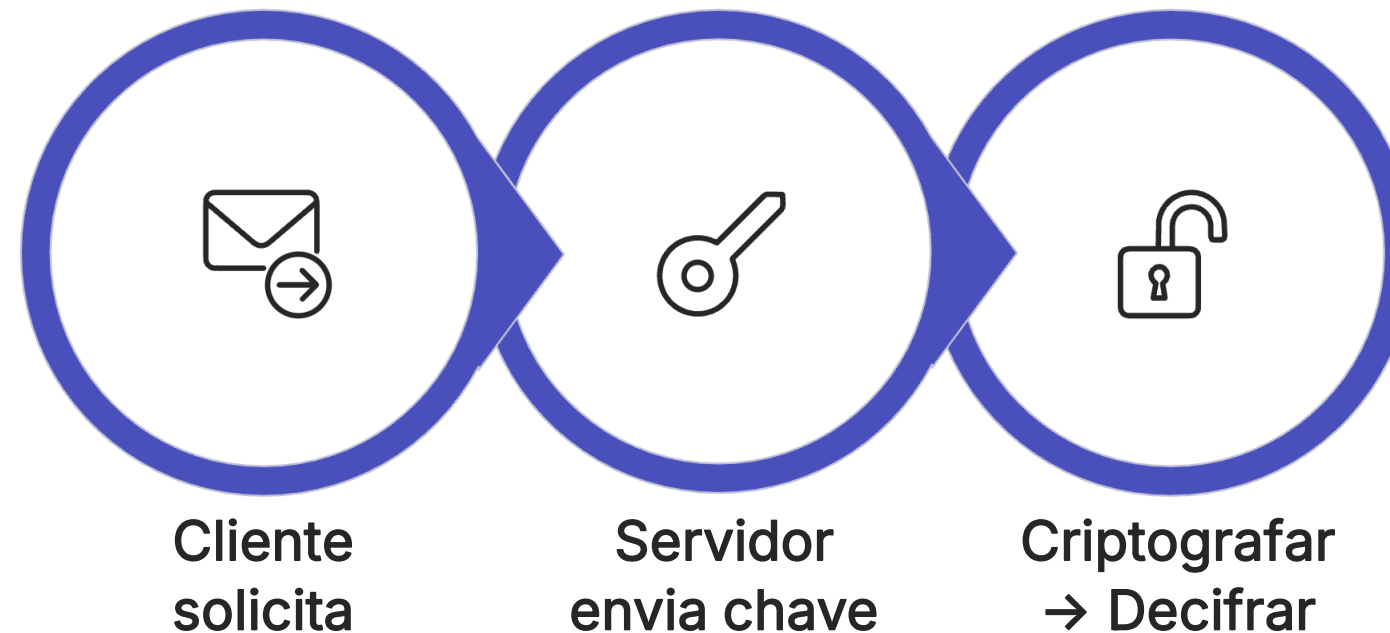
Mesma chave para cifrar e decifrar. Rápida e eficiente para grandes volumes de dados.

- Algoritmo padrão: **AES** (Advanced Encryption Standard)
- Ideal para dados em repouso e comunicações internas
- Desafio: compartilhar a chave com segurança

🔒 Criptografia Assimétrica

Par de chaves: **chave pública** (cifra) e **chave privada** (decifra). Mais segura para troca de chaves.

- Algoritmo padrão: **RSA**
- Base do HTTPS e dos certificados digitais
- Garante troca segura mesmo em canais abertos



Na prática, o HTTPS combina ambas: a assimétrica troca a chave com segurança, e a simétrica é usada para cifrar o tráfego pela velocidade.

O que é Hash?

Hash é uma **função matemática unidirecional**: transforma qualquer entrada em uma sequência de caracteres de tamanho fixo. Ao contrário da criptografia, o processo é **irreversível** — não existe "chave" para reverter.



Irreversível

Não é possível recuperar o dado original a partir do hash gerado.



Determinístico

A mesma entrada sempre gera exatamente o mesmo hash.



Efeito Avalanche

Uma pequena mudança na entrada gera um hash completamente diferente.

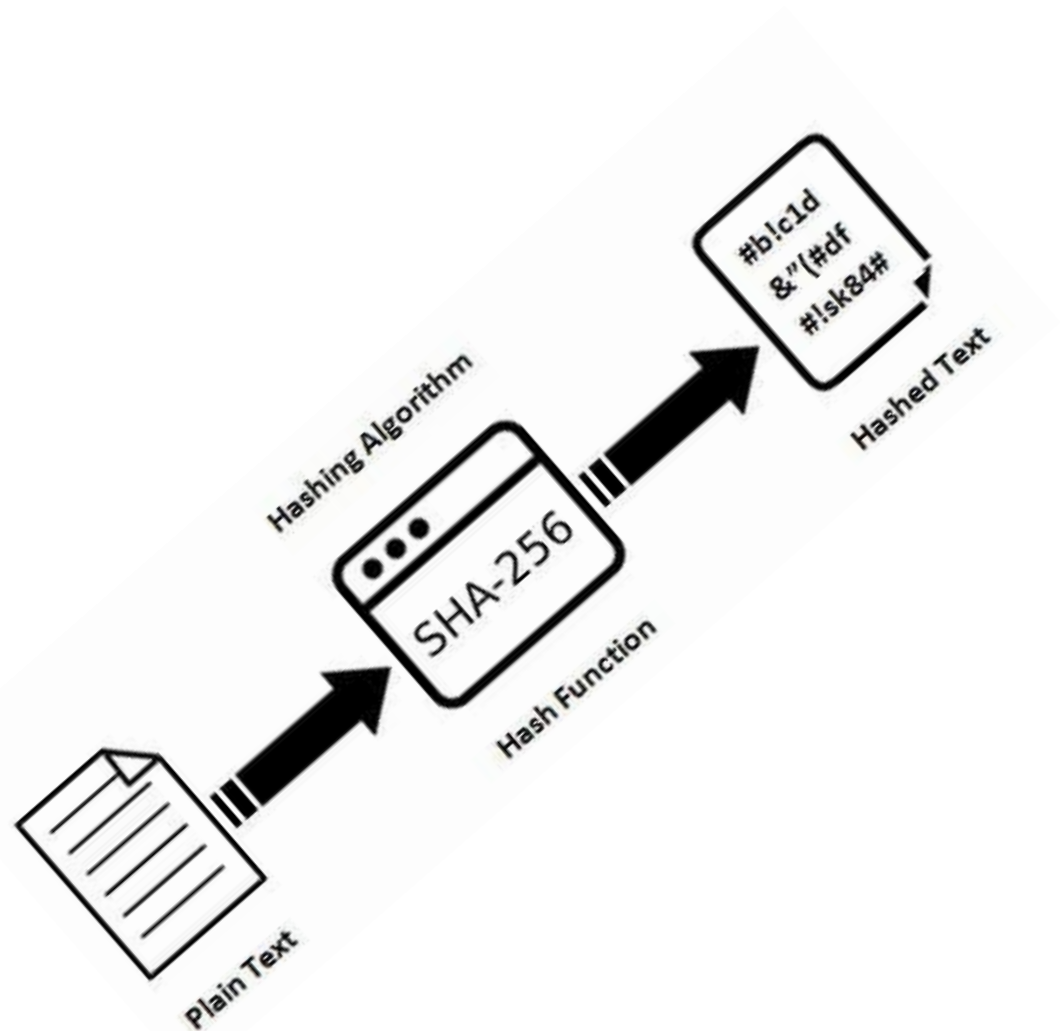


Valida Integridade

Confirma se um arquivo ou dado foi alterado, sem revelar seu conteúdo.

Algoritmos mais usados: SHA-256 e SHA-512. Aplicações:

- armazenamento de senhas, assinaturas digitais e verificação de integridade de arquivos.



Comparativo Essencial

Criptografia ≠ Hash

Esses dois conceitos são frequentemente confundidos, mas têm propósitos e características completamente distintos. Entender a diferença é fundamental para aplicar cada um corretamente.

Característica	 Criptografia	 Hash
Reversível?	✓ Sim, com a chave correta	✗ Não — unidirecional
Usa chave?	✓ Sim (simétrica ou assimétrica)	✗ Não necessita de chave
Objetivo principal	Proteger o conteúdo em sigilo	Validar integridade / fingerprint
Tamanho da saída	Variável (proporcional à entrada)	Fixo (ex.: SHA-256 = 64 hex chars)
Uso com senhas	Não recomendado para senhas	✓ Padrão para armazenar senhas

👉 **Boa prática:** Senhas devem ser armazenadas com hash (preferencialmente com salt), nunca com criptografia reversível. Se o banco de dados for comprometido, hashes não revelam as senhas originais.

HTTPS na Prática

O que o HTTPS protege?

- ✓ Headers da requisição
- ✓ Payload (corpo dos dados)
- ✓ Cookies de sessão
- ✓ Tokens de autenticação
- ✓ Confidencialidade end-to-end

Como funciona?

HTTPS = HTTP + **TLS** (Transport Layer Security). O TLS estabelece um canal criptografado entre cliente e servidor antes de qualquer dado ser trafegado.

O processo de handshake TLS usa criptografia assimétrica para trocar chaves e, em seguida, simétrica para o tráfego — garantindo velocidade com segurança.

Cliente envia
Requisição HTTP para
o servidor.

API responde
Resposta segura e
autenticada.



TLS estabelece
Canal cifrado entre as
partes.



Regra absoluta: APIs modernas devem obrigatoriamente usar HTTPS. Nunca exponha endpoints em HTTP puro em produção.

Auditoria e Rastreabilidade

Rastreabilidade é a **capacidade de identificar quem fez o quê, quando e como** em um sistema. Um log de auditoria bem estruturado é indispensável para segurança, conformidade legal e resolução de incidentes.



1

Quem?

Identificação do usuário autenticado (ID, e-mail, perfil).

2

Quando?

Data e hora exata da ação, preferencialmente em UTC.

3

O quê?

Ação executada: criação, alteração, exclusão, login, falha.

4

Como?

IP de origem, dispositivo, recurso alterado e resultado da operação.



Exemplos práticos de eventos a registrar: Alteração de cadastro, exclusão de registro, login bem-sucedido, falha de autenticação, tentativa de acesso não autorizado.

Controle de Acesso

Princípio do Menor Privilégio

O Princípio do Menor Privilégio (Least Privilege) determina que cada usuário, processo ou serviço deve receber **somente o acesso estritamente necessário** para desempenhar sua função — nada além disso.

Acesso excessivo amplia a superfície de ataque. Se uma conta comprometida tiver permissões administrativas desnecessárias, o impacto de uma falha é muito maior.

Aluno

✓ Consultar notas e materiais. ✗ Lançar ou editar dados.

Professor

✓ Lançar e editar notas da turma. ✗ Acessar dados administrativos.

Admin

✓ Gestão completa do sistema. Acesso auditado e restrito.

⚠ Revisões periódicas de permissões são obrigatórias. Colaboradores que mudam de função ou saem da empresa devem ter acessos imediatamente revisados ou revogados.

Vulnerabilidades Comuns em Integrações

⊘ Nunca Faça

- ✗ Expor token em URL ou logs
- ✗ Armazenar segredos no código-fonte
- ✗ Registrar senhas em logs
- ✗ Trafegar dados em HTTP puro
- ✗ Permitir acesso irrestrito a endpoints
- ✗ Ignorar expiração de tokens/sessão
- ✗ Confiar apenas na validação do front-end

☑ Sempre Faça

- ✓ Validação server-side em toda requisição
- ✓ Logs de auditoria estruturados
- ✓ HTTPS obrigatório em produção
- ✓ Aplicar Least Privilege
- ✓ Configurar expiração de sessão
- ✓ Revisão periódica de acessos
- ✓ Usar variáveis de ambiente para segredos

Exemplo Conceitual

Headers Seguros vs. Inseguros

A forma como as credenciais são transmitidas nos headers HTTP revela muito sobre o nível de segurança de uma integração. Veja a diferença entre uma implementação correta e uma perigosa:

✓ ☒ Header Seguro — Correto

Authorization: Bearer token_seguro

Token JWT ou opaco, de tempo limitado, gerado pelo servidor de autenticação. Não contém dados sensíveis diretamente legíveis.

✗ ☐ Header Inseguro — Jamais Faça

Authorization: senha123

Credencial em texto puro diretamente no header. Qualquer interceptação ou log expõe imediatamente o acesso da conta.

⚠ **Regra absoluta:** Credenciais (senhas, tokens de API, chaves secretas) jamais devem ser expostas em texto puro em headers, URLs, logs ou código-fonte. Use tokens de curta duração e revogue imediatamente em caso de vazamento.

DESENVOLVIMENTO DE SOFTWARE SEGURO



Dimensão Ética

Responsabilidade Profissional no Tratamento de Dados

Ao desenvolver sistemas integrados, o profissional de TI lida cotidianamente com informações altamente sensíveis de pessoas reais. Cada linha de código tem implicações que vão além do técnico.



Identidade

CPF, RG, endereço, data de nascimento



Dados Financeiros

Salário, contratos, extratos e operações



Saúde

Prontuários, diagnósticos, medicamentos



Contratos

Acordos legais, termos e informações jurídicas

Segurança é responsabilidade técnica ou ética? A resposta é: Ambas. A LGPD (Lei Geral de Proteção de Dados) torna isso juridicamente vinculante — falhas de segurança podem gerar sanções legais para empresas e profissionais.

Atividade Guiada

Análise de Sistema Fictício

A equipe recebeu o relatório de auditoria do portal corporativo da Situação de Aprendizagem. Sua tarefa é analisar cada achado, classificar a vulnerabilidade e propor a correção técnica adequada.

→ Identifique as Falhas

Leia o relatório e liste: exposição de dados, ausência de logs, comunicação insegura, armazenamento incorreto e excesso de privilégio.

→ Proponha Melhorias

Para cada falha, descreva a solução técnica: qual prática aplicar, qual tecnologia usar e como implementar.

→ Classifique por Risco

Atribua criticidade (Alta / Média / Baixa) a cada vulnerabilidade encontrada com base no impacto potencial.

→ Apresente ao Grupo

Compartilhe os achados e compare com as análises dos colegas. Discuta as diferentes abordagens de solução.

Questões para Aprofundamento

Discuta com a turma. Não há apenas uma resposta certa — o objetivo é desenvolver o raciocínio técnico crítico sobre segurança em integrações reais.

1

Criptografia ou Hash?

Para armazenar senhas de usuários, qual abordagem é correta e por quê? O que muda se precisarmos exibir a senha ao usuário?

2

Quem acessa os logs?

Os logs de auditoria contêm informações sensíveis. Como protegê-los? Quem deve ter permissão de leitura?

3

Token vazou — e agora?

Quais são os passos imediatos? Como a expiração de tokens mitiga o impacto? O que muda com refresh tokens?

4

HTTPS basta?

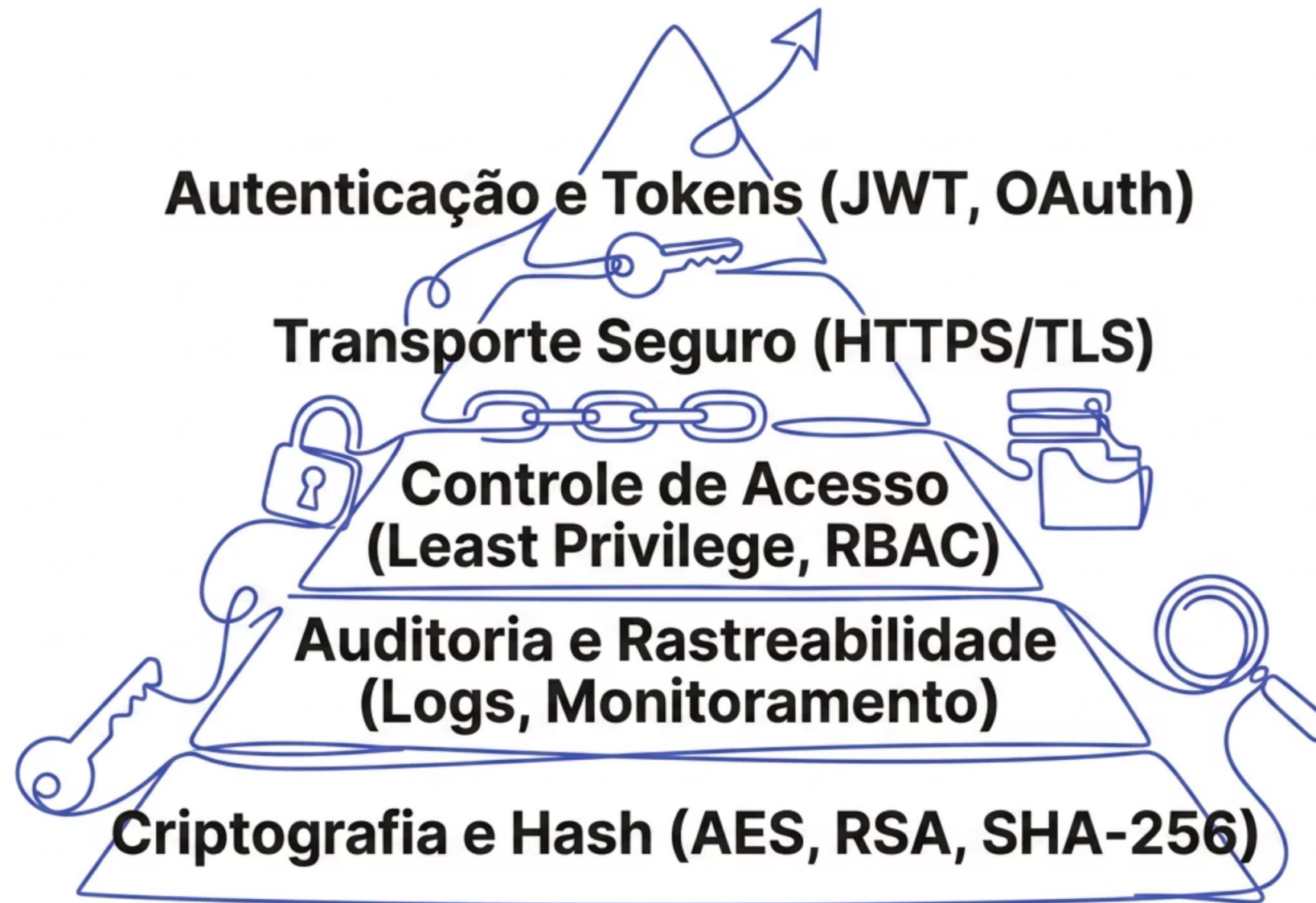
HTTPS protege o transporte, mas e os dados em repouso no banco? E os logs? Quais outras camadas são necessárias?

5

Front-end garante segurança?

Um formulário com validação no Angular é suficiente? Por que a validação server-side é insubstituível?

Camadas de Segurança em uma Integração



A segurança de uma integração é construída em camadas complementares. Nenhuma camada, isolada, é suficiente. A falha em qualquer nível compromete as demais — por isso a abordagem deve ser holística e contínua.

Segurança como Obrigação Legal

O que diz a LGPD?

A Lei Geral de Proteção de Dados (Lei nº 13.709/2018) estabelece que organizações e profissionais que tratam dados pessoais devem adotar medidas técnicas e administrativas para protegê-los.

- Finalidade: dados usados apenas para o fim declarado
- Necessidade: coletar somente o mínimo necessário
- Segurança: proteger contra acessos não autorizados
- Responsabilização: demonstrar conformidade com evidências

Multas

Até 2% do faturamento anual, limitado a R\$ 50 milhões por infração.

Bloqueio

Suspensão ou proibição do tratamento de dados pessoais.

Publicização

Divulgação pública da infração — dano grave à reputação.

 Como desenvolvedor, você é parte da cadeia de responsabilidade. Implementar as boas práticas desta aula não é apenas técnico — é um dever legal e ético.

Evidência de Competência

O que você é capaz de fazer agora

Ao concluir esta aula, você desenvolveu as seguintes competências profissionais, alinhadas ao perfil de egresso do Curso Técnico em Desenvolvimento de Sistemas:



✓ Explicar e aplicar criptografia simétrica e assimétrica

Identificar qual algoritmo (AES, RSA) usar em cada contexto de integração.



✓ Diferenciar criptografia de hash com precisão técnica

Escolher a abordagem correta para cada necessidade: sigilo vs. integridade.



✓ Compreender e exigir HTTPS em integrações

Entender o papel do TLS e por que nenhuma API em produção deve operar sem ele.



✓ Implementar auditoria, rastreabilidade e menor privilégio

Projetar sistemas com logs estruturados e controle de acesso baseado em perfil.



✓ Atuar com responsabilidade ética e legal no tratamento de dados

Reconhecer as implicações da LGPD e das boas práticas no exercício profissional.

Segurança é Profissionalismo

Construir sistemas seguros não é uma tarefa extra — é parte essencial do trabalho de um desenvolvedor qualificado. Cada decisão técnica sobre criptografia, hash, logs e controle de acesso reflete o compromisso com a qualidade, a ética e a responsabilidade profissional.

Proteja os Dados

Use criptografia e hash corretamente. Nunca armazene ou trafegue dados sensíveis sem proteção.

Registre Tudo

Logs de auditoria são a memória do sistema. Rastreabilidade é inegociável.

Controle o Acesso

Aplique o princípio do menor privilégio. Revise permissões periodicamente.

Aja com Ética

Você trabalha com dados de pessoas reais. A responsabilidade é técnica e humana.

